

Building a Mini ChatGPT

A Complete Learning Journal with nanoGPT

Brice Sommers | May 2026

PLAN

Step 1 — Set Up Your Environment

- Install Python, Pytorch and NanoGPT.

Step 2 — How GPT Works (Plain English)

- Tokenization, Transformers and prediction.

Step 3 — Train Your First Model

- Run NanoGPT on Shakespeare

Step 4 — Understanding the Code

- Code review.

Step 5 — Building the Instruct (Chat) Model

- Fine-tune model on Q&A.

Step 1 — Set Up Your Environment

Goal

Install all the tools needed to train a GPT model. This includes Python, PyTorch with MPS support, and the nanoGPT repository.

1.1 Check Python Version

First, verify Python is installed and meets the minimum version requirement (3.10+).

Command:

```
python3 --version
```

1.2 Create a Conda Environment

Rather than installing packages system-wide, we create an isolated Conda environment called 'nanogpt'. The (base) prefix in the terminal confirmed Conda was already installed.

Command:

```
conda create -n nanogpt python=3.11 -y
```

Activate the environment:

```
conda activate nanogpt
```

1.3 Install Required Packages

Install PyTorch and all supporting libraries needed for training.

Command:

```
pip install torch numpy transformers datasets tiktoken wandb tqdm
```

1.4 Clone the nanoGPT Repository

Download Andrej Karpathy's nanoGPT codebase from GitHub.

Command:

```
git clone https://github.com/karpathy/nanoGPT.git  
cd nanoGPT
```

1.5 Verify GPU Support (MAC Only)

Check that PyTorch can see and use the Apple Silicon GPU via the MPS (Metal Performance Shaders) backend.

Command:

```
python -c "import torch; print(torch.__version__);  
print(torch.backends.mps.is_available())"
```

Step 1 Summary

- Python 3.13.7 confirmed
- Conda environment 'nanogpt' created with Python 3.11
- PyTorch 2.12.0 and all dependencies installed
- nanoGPT repository cloned
- Apple M1 GPU confirmed ready (MPS: True)

To resume work in future sessions, always run:

```
conda activate nanogpt  
cd nanoGPT
```

Step 2 — How GPT Works (Plain English)

Goal

Build a mental model of how GPT works before touching any code. No math required.

2.1 Tokenization

GPT does not read letters or full words. It reads tokens, which are chunks of text. Common words are one token; rare words are split into multiple tokens. GPT-2 uses a vocabulary of about 50,000 tokens.

Each token is converted into a number so the model can process it mathematically. For example:

```
Hello -> 9906
,      -> 11
I      -> 40
am     -> 309
```

2.2 Prediction

The only job of a GPT model is to predict the next token given all previous tokens. Nothing more. It does this over and over, with each new token becoming part of the input for the next guess.

Example: given 'The cat sat on the', the model predicts 'mat' with 72% probability. This is the entire intelligence of GPT. Do it billions of times on enough text and something remarkable emerges.

2.3 The Transformer

Tokens pass through stacked layers called transformer blocks. Each layer refines the understanding:

- Layer 1 — basic patterns (common letter sequences)
- Layer 4 — word relationships (which words go together)
- Layer 8 — grammar and structure
- Layer 12 — final prediction

Each layer uses attention, a mechanism that lets the model look at all other tokens to decide which ones are relevant. nanoGPT uses 6 layers. GPT-4 uses hundreds.

2.4 Training

Training works by repeatedly showing the model text examples and nudging it to do better:

- Feed text in (Shakespeare, web pages, books)
- Make a prediction for the next token
- Measure how wrong it was (the loss)
- Adjust the weights slightly to reduce the error
- Repeat billions of times

The weights are millions of numbers inside the model. After enough training, those numbers encode an incredible amount of knowledge about language.

Step 3 — Train Your First Model

Goal

Train a character-level GPT model on all of Shakespeare's works and generate Shakespeare-style text.

3.1 Prepare the Shakespeare Dataset

Command:

```
python data/shakespeare_char/prepare.py
```

3.2 Train the Model

Train using the M1 GPU by specifying `--device=mps`. The config file sets up a 10.65M parameter model with 6 layers.

Command:

```
python train.py config/train_shakespeare_char.py --device=mps
```

Key config settings:

```
n_layer = 6           # 6 transformer layers
n_head = 6             # 6 attention heads
n_embd = 384           # embedding size
block_size = 256       # context window
max_iters = 5000       # training steps
batch_size = 64        # examples per step
```

3.3 Generate Text from the Trained Model

Command:

```
python sample.py --out_dir=out-shakespeare-char --device=mps
```

Step 4 — Understanding the Code

Goal

Understand what the two core files of nanoGPT actually do. The entire codebase is only ~600 lines total.

4.1 model.py (~300 lines) — The Brain

This file defines the neural network architecture. It contains four classes that build on each other like Lego:

CausalSelfAttention

The attention mechanism. Lets the model look at all previous tokens to decide which are relevant for the next prediction. 'Causal' means it can only look backwards, not forwards (otherwise it would be cheating).

MLP (Multi-Layer Perceptron)

A simple two-layer network that processes each token after attention. Think of it as the model 'thinking' about what it just noticed.

Block

One full transformer layer. It is simply: attention + MLP stacked together. nanoGPT stacks 6 of these blocks.

GPT

The complete model. It combines:

- An embedding layer (converts tokens to vectors of numbers)
- All 6 transformer blocks
- A final linear layer that outputs probabilities for the next token

4.2 train.py (~300 lines) — The Teacher

This file runs the training loop. The core training logic is just 4 lines:

```
X, Y = get_batch('train')      # grab a batch of text
logits, loss = model(X, Y)      # make predictions, measure error
loss.backward()                 # figure out what to fix
optimizer.step()                # nudge the weights
```

Everything else in train.py is bookkeeping: saving checkpoints every 250 steps, logging the loss, adjusting the learning rate over time, and loading/saving model weights.

Step 5 — Building the Instruct (Chat) Model

Goal

Fine-tune the model on conversation data so it learns to respond like a chatbot. This is the same technique used to turn GPT-3 into ChatGPT.

Key Concept: Base Model vs Instruct Model

The Shakespeare model is a base model. It learned to complete text, but has no concept of answering questions. If you asked it 'What is the capital of France?' it might just generate more questions.

An instruct model is the same architecture, but fine-tuned on conversation data formatted as Human/Assistant pairs. It learns the pattern: Human asks, Assistant answers.

The real ChatGPT was built exactly this way: GPT-3 (base) -> fine-tuned on conversations -> ChatGPT (instruct).

We will build a new NanoGPT chatbot LLM called NanoGPT2 from the base model GPT-2.

5.1 Installation

Clone the repo.

Command:

```
git clone https://github.com/bricesommers/nanoGPT2
cd nanoGPT2
pip install -r requirements.txt
```

Create a virtual environment.

Command:

```
python3 -m venv .venv
source .venv/bin/activate
```

Install dependencies.

Command:

```
pip install -r requirements.txt
```


5.2 Conversation Dataset

A 50-conversation dataset was created covering topics including science, philosophy, AI, programming, and general knowledge. Each exchange uses a consistent format:

```
Human: What is artificial intelligence?
Assistant: Artificial intelligence is the field of building computer
systems that can perform tasks that normally require human
intelligence,
like understanding language, recognizing images, or making decisions.

Human: What is machine learning?
Assistant: Machine learning is a branch of AI where computers learn
from data instead of being explicitly programmed...
```

The dataset is the data/input.txt file.

5.3 Fine-tune

Use input.txt to fine-tune GPT-2 so we can have a conversation with it.

Command:

```
python finetune.py --data data/input.txt --epochs 5 --output out/
```

Options.

Command:

```
--data      path to your input.txt (default: data/input.txt)
--output    where to save the model (default: out/)
--epochs    number of training epochs (default: 5)
--lr        learning rate (default: 5e-5)
--batch_size batch size (default: 2)
```

5.4 Chat

Chat with the model.

Command:

```
python chat.py --model out/
```

Options.

Command:

```
--model      path to fine-tuned model (default: out/)
--temperature randomness of responses, 0.1-1.0 (default: 0.7)
--max_tokens  max length of each response (default: 150)
```